

DiffusionPack: Bin Packing with Custom Human Preferences

Abstract—The three-dimensional bin packing problem (3D-BPP) is fundamental in robotic palletizing and warehouse automation, where systems must generate compact, feasible layouts that also satisfy user-defined requirements. Existing exact and heuristic methods scale poorly or cannot accommodate dynamic preferences, while learning-based approaches typically require retraining for new constraints. We present DiffusionPack, a planning framework that formulates 3D-BPP as a conditional generation problem and learns the joint distribution of complete packing layouts using a diffusion model. To ensure physical feasibility, diffusion outputs are refined through a projection mechanism guided by point-based heuristics. The method incorporates user guidance at inference: hard constraints are enforced through inpainting, and soft preferences are integrated via guidance functions. Natural language instructions are supported through an LLM that translates user input into constraint specifications. Real-world validation on a Franka robot demonstrates reliable execution, highlighting the applicability of our approach to practical robotic packing and palletizing tasks. Our implementation is open-sourced at: [DiffusionPack](#)

Index Terms—Integrated Planning and Learning, AI and Machine Learning in Manufacturing and Logistics Systems, Human-Centered Robotics

I. INTRODUCTION

Robotic palletizing, warehouse automation, and autonomous packing systems increasingly rely on algorithms that can generate compact, safe, and user-compliant 3D packing strategies. At the core of these applications lies the three-dimensional bin packing problem (3D-BPP), a classical and challenging combinatorial optimization problem. The task involves arranging a set of cuboid items inside a container, typically with the goal of maximizing packing density. The problem appears in logistics, manufacturing, distribution centers, and automated fulfillment systems. Two major variants exist: *online* bin packing, where items arrive sequentially and must be placed immediately, and *offline* bin packing, where all items are known prior, enabling global optimization but substantially enlarging the search space.

In this work, we concentrate on the offline setting with a single container. While the primary objective is high space utilization, real-world robotic packing systems often operate under additional constraints. Users may impose strict spatial requirements, such as placing fragile or high-value items only on top, or may specify soft preferences, such as grouping similar items together or placing certain boxes early or late in the sequence. These requirements are common in practice but difficult to express in traditional bin-packing formulations. Existing solvers lack an efficient mechanism for incorporating such preferences dynamically at test time. Our work addresses this gap by enabling user-specified constraints to be integrated without retraining the model.

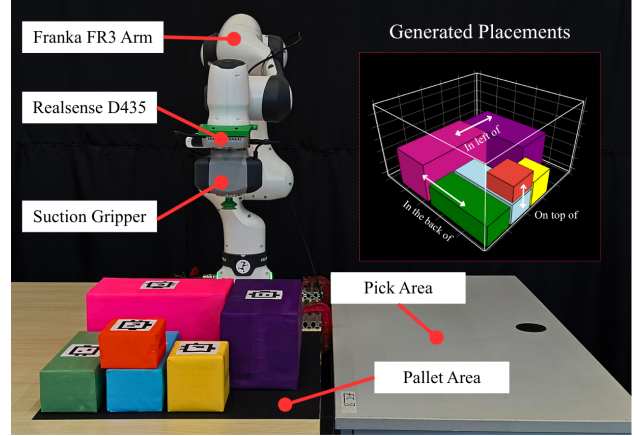


Fig. 1: **DiffusionPack** generates compact, high-density packing configurations that satisfy user-defined spatial preferences (e.g., **Pink** left of **Purple** & **Pink** behind **Green** & **Red** above **Blue**).

Classical exact methods such as branch-and-bound [1] and mixed-integer programming yield optimal solutions but scale poorly with problem size and struggle to handle custom, user-defined constraints. As a result, the field relies heavily on heuristic and approximation algorithms [2], [3], which are faster but make local decisions and process each instance independently, limiting their ability to capture global structure or adapt to diverse preferences.

Learning-based bin packing approaches [4], [5] have demonstrated improvements under complex constraints such as stability, but these systems remain limited in flexibility. Reinforcement learning methods are challenging to train in high-dimensional combinatorial spaces and must be retrained to incorporate new constraints. More fundamentally, modeling diverse human preferences and generalizing to unseen specifications remain open challenges for existing learning-based pipelines. In contrast, our approach incorporates preferences directly during inference, offering both flexibility and generalization without modifying the trained model.

We introduce **DiffusionPack**, a diffusion-based framework that learns the distribution of 3D packing layouts from supervised data. We formulate the 3D-BPP as a conditional generation task in which the model predicts the joint centroid locations of all items given their dimensions. This differs from prior learning-based techniques, which generally decompose the problem into item sequencing followed by position selection. Joint generation captures global spatial structure more effectively and avoids compounding suboptimal decisions across stages.

Although diffusion models generate coherent global lay-

outs, their predictions may lack precise physical feasibility. Traditional heuristics, in contrast, excel at producing valid, boundary-aligned placements but often yield suboptimal layouts when used alone. We combine these advantages through a hybrid pipeline that projects diffusion-generated predictions onto feasible configurations using point-based heuristics. This projection step ensures physical plausibility while preserving the spatial efficiency through diffusion.

This formulation naturally supports user preferences at inference time. Hard constraints are enforced through in-painting, while guidance functions softly steer the diffusion process toward satisfying user-defined objectives. We further demonstrate that natural language instructions can be mapped to constraint specifications using a large language model (LLM), enabling intuitive preference-conditioned packing for human-robot collaboration. We train our system on a cutting stock dataset [6] and evaluate its generalization ability on out-of-distribution data and in real robotic experiments.

Our key contributions are

- A hybrid diffusion-projection planner that predicts complete 3D packing configurations while ensuring physical feasibility.
- A flexible inference-time mechanism for incorporating human preferences without retraining.
- Integration of natural language conditioning through LLM-generated constraints, enabling intuitive human-robot interaction.

II. RELATED WORKS

A. Offline Bin Packing

1) *Exact and Heuristic Methods.*: Classical approaches for offline three-dimensional bin packing rely on combinatorial search or mixed-integer optimization [7]. Exact methods, including branch-and-bound formulations [1] and Integer Linear Programming (ILP) variants [8], [9], [10], explore the search space exhaustively and can yield optimal packings when the number of items is small. However, their computational cost grows rapidly with problem size, limiting their applicability in real-world settings.

To address scalability, a broad class of heuristic methods has been developed. Extreme-point strategies [11], block-building schemes [12], and constructive heuristics such as first-fit and best-fit decreasing [2], [3] place items sequentially using local rules. Metaheuristic approaches, including genetic algorithms [13], [14], tabu search [15], and guided local search, explore larger neighborhoods around these constructive solutions. Heuristics generate feasible packings efficiently but rely on hand-crafted rules, limiting adaptability to task-specific constraints, relational preferences, and dynamic requirements. They also process instances independently, ignoring global spatial regularities.

2) *Learning-Based Methods.*: Recent learning-based methods enhance packing quality and scalability by learning data-driven policies or value functions. Reinforcement learning approaches [16], [17], [5] typically adopt a two-stage process separating item sequencing and placement. Ranked Reward [6] leverages self-play to handle large action spaces,

while multimodal architectures such as [18] incorporate visual and geometric signals. RCQL [19] extends this direction by introducing quantile-based value estimation to stabilize learning under sparse rewards. However, these approaches still rely on masked discrete action spaces and a separate sequencing and placement pipeline, which creates error propagation, limits global spatial reasoning, and constrains the modeling of user-defined preferences.

Generative models have recently emerged as a promising alternative for structured planning. GAN-assisted evolutionary methods [20] enhance exploration, and diffusion models have been investigated for combinatorial problems such as the Travelling Salesman Problem (TSP) and Maximum Independent Set (MIS) [21], [22]. In robotics, diffusion-based rearrangement frameworks [23], [24], [25] demonstrate that diffusion models can implicitly capture spatial regularities, while GFPack [26] addresses 2D irregular packing using gradient fields. However, these works either operate on discrete structures or low-dimensional geometric tasks, do not extend to full 3D packing with feasibility constraints, and lack mechanisms for integrating human preferences at inference time.

B. Bin Packing with Custom Preferences

Beyond geometric compactness, several works incorporate auxiliary constraints such as stability, load balancing, or stacking rules. Rule-based methods impose precedence or hierarchical constraints [27], while mixed-integer formulations have been used for load balancing and product-family considerations [28]. Learning-based models have encoded stacking or stability restrictions [29], [30], and compositional diffusion frameworks [31] optimize energy-based preference functions. However, StructDiffusion [23] and DiffusionCCSP [31] typically rely on fixed constraint structures, scale only to small object sets, and require retraining when new preference types are introduced.

Real robotic packing often involves dynamic, user-specific requirements such as relational rules, grouping preferences, and task-dependent priorities. Existing methods cannot flexibly handle such heterogeneous constraints at inference. Our approach supports both hard and soft human preferences expressed in natural language.

Positioning Existing bin packing methods across exact, heuristic, and learning-based paradigms face fundamental limitations. Exact and ILP approaches lack adaptability; heuristics depend on rigid instance-specific rules, and learning methods separate ordering and placement, limiting global reasoning. **DiffusionPack** addresses these gaps through unified 3D layout generation, projection-based feasibility refinement, and preference integration via natural language input.

III. METHODOLOGY

We consider the offline single-bin 3D bin packing problem. The container is an axis-aligned cuboid with dimensions (L, W, H) . A fixed set of N cuboid items is given, where item i has dimensions (l_i, w_i, h_i) and centroid $\mathbf{p}_i = (x_i, y_i, z_i) \in \mathbb{R}^3$. Rotations are not considered.

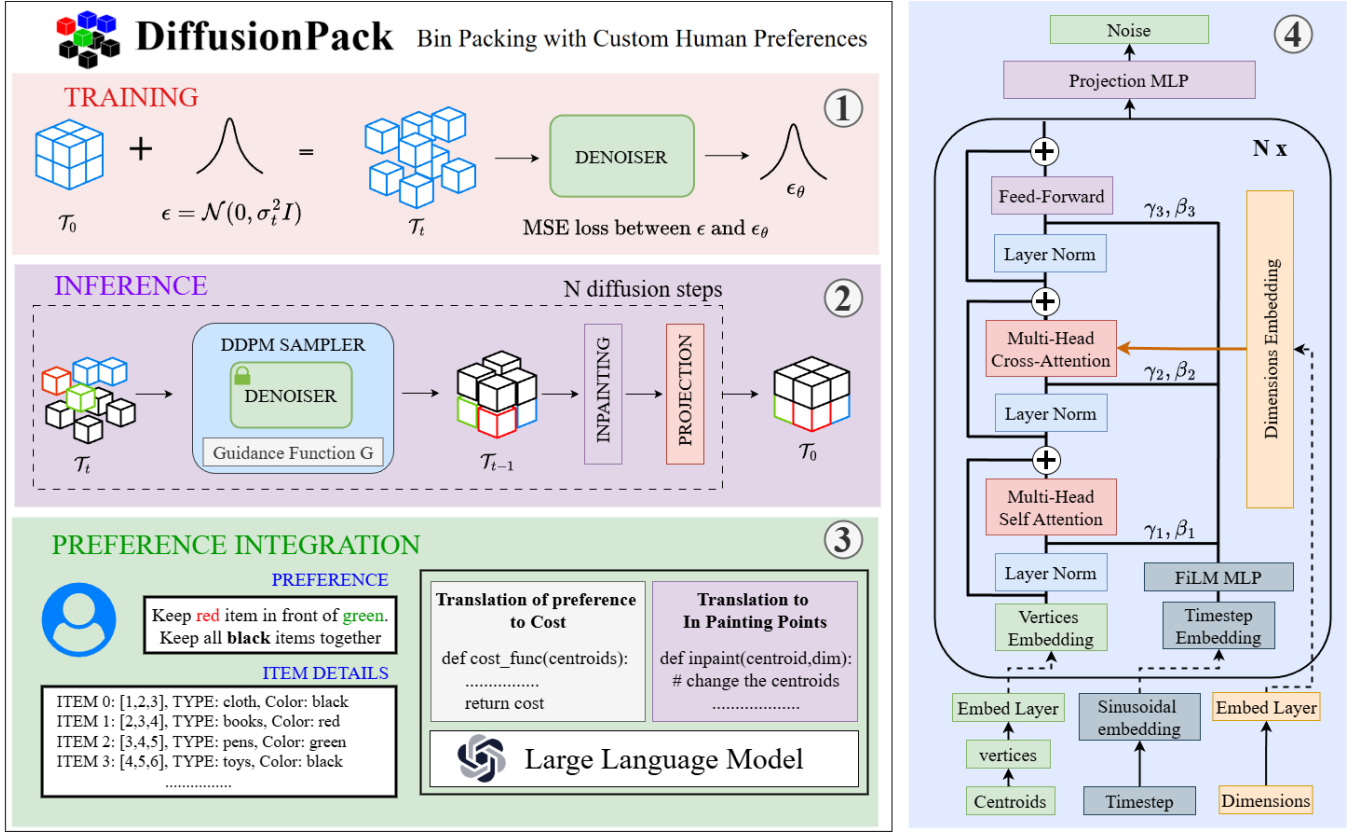


Fig. 2: Overview of **DiffusionPack**: (1) **Training**: The diffusion model learns to denoise cuboid centroids with MSE loss. (2) **Inference**: Candidate packings are generated through iterative denoising, projection, and inpainting. (3) **Preference Integration**: User preferences expressed in natural language are translated by an LLM into cost functions and inpainting rules. (4) **Denoiser Architecture**: An attention-based network enabling context-aware permutation-invariant packing.

The decision variables are the centroid positions $\{\mathbf{p}_i\}_{i=1}^N$. A placement is feasible if:

- (i) *Boundary constraints*: $\frac{l_i}{2} \leq x_i \leq L - \frac{l_i}{2}$, $\frac{w_i}{2} \leq y_i \leq W - \frac{w_i}{2}$, $\frac{h_i}{2} \leq z_i \leq H - \frac{h_i}{2}$, $\forall i$.
- (ii) *Non-overlap constraints*: for all $i \neq j$, $|x_i - x_j| \geq \frac{l_i+l_j}{2} \vee |y_i - y_j| \geq \frac{w_i+w_j}{2} \vee |z_i - z_j| \geq \frac{h_i+h_j}{2}$.

The general objective is to maximize packing efficiency $\eta = \frac{\sum_{i=1}^N l_i w_i h_i}{LWH}$ subject to the above constraints. Additionally, we enforce a stability constraint requiring each cuboid to have at least 60% of its base area supported.

We cast this as a conditional generative modeling problem: a diffusion model jointly produces $\{\mathbf{p}_i\}_{i=1}^N$ given the item dimensions, and is trained to replicate expert placements.

A. Diffusion Model For Our Approach

Diffusion models learn complex data distributions by gradually corrupting data samples into Gaussian noise and then learning to reverse this process. The centroid configuration $\mathbf{x}_0 = \{\mathbf{p}_1, \dots, \mathbf{p}_N\}$, where $\mathbf{p}_i \in \mathbb{R}^3$, is modeled using a denoising diffusion probabilistic framework. A standard Gaussian forward process progressively perturbs clean centroids with noise according to a predefined variance schedule $\{\beta_t\}_{t=1}^T$. Using the closed-form formulation, noisy samples can be obtained directly as $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$, where $\epsilon \sim \mathcal{N}(0, I)$ and $\bar{\alpha}_t = \prod_{i=1}^t (1 - \beta_i)$.

The reverse process learns to reconstruct structured centroid layouts from noise while conditioning on cuboid dimensions $\mathbf{c} = \{\mathbf{c}_1, \dots, \mathbf{c}_N\}$, with $\mathbf{c}_i \in \mathbb{R}^3$. Instead of directly predicting clean centroids, the network predicts the injected noise $\epsilon_\theta(\mathbf{x}_t, \mathbf{c}, t)$. The training objective minimizes $\mathcal{L}(\theta) = \mathbb{E}_{t, \mathbf{x}_0, \epsilon} [\|\epsilon - \epsilon_\theta(\mathbf{x}_t, \mathbf{c}, t)\|_2^2]$, where t is uniformly sampled from diffusion steps.

During inference, generation starts from Gaussian noise and iteratively applies the learned reverse update to obtain a clean centroid set $\mathbf{x}_0$. The sampling update rule is

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, \mathbf{c}, t) + \sqrt{\bar{\beta}_t} \mathbf{z} \right) \quad (1)$$

with $\mathbf{z} \sim \mathcal{N}(0, I)$. The denoiser is implemented using an attention-based architecture (Fig. 2(4)), enabling global reasoning over all items and capturing spatial dependencies among cuboids. Training and inference procedures are summarized in Fig. 2(1-2).

B. Diffusion-Guided Packing With Projections

While diffusion models generate samples consistent with the data distribution, they do not guarantee strict feasibility. Projected Diffusion Models [32] address this by applying a projection operator after each denoising step, defined as $P_C(x) = \arg \min_{y \in C} \|y - x\|_2^2$, which maps any point x to its nearest feasible counterpart in the constraint set C .

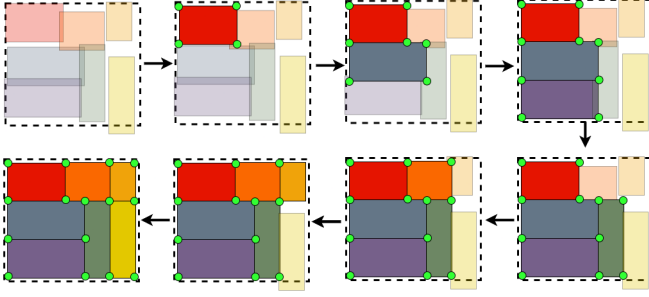


Fig. 3: Step-by-step 2D illustration of projection using keypoints. Keypoints are shown in green, projected cuboids in dark, and remaining cuboids in light.

Incorporating this into the reverse diffusion modifies the update in Eq. 1.

$$\mathbf{x}_{t-1} = P_C \left(\frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \varepsilon_\theta(\mathbf{x}_t, \mathbf{c}, t) + \sqrt{\beta_t} \mathbf{z} \right) \right) \quad (2)$$

While [32] relies on gradient-based projections, we propose a projection operator that aligns predicted centroids with discrete packing keypoints. These keypoints [11], [33] are widely used in classical bin packing heuristics to give quick feasible placements. The stochastic nature of Diffusion Models can lead to mild interpenetrations or boundary violations even when trained on valid data. By projecting noisy predictions to the nearest feasible keypoints, our method ensures valid placements. Section IV-D includes ablations on keypoint selection and projection frequency, while Algorithm 1 provides a detailed description. A two-dimensional illustration is shown in Fig. 3.

After generating multiple candidate configurations from the diffusion model, we perform a stabilization step that iteratively projects cuboids downward until they lie on the container floor or on top of previously placed items. Cuboids whose base is supported by less than 60% of its area are discarded to ensure physical plausibility. Only configurations satisfying stability and constraint criteria are retained; those violating spatial or preference conditions are discarded. The remaining candidates are ranked according to the selected objective, which can be either maximizing packing density or maximizing the number of placed items. The complete algorithm is detailed in Algorithm 2

C. Applying Preferences for Human-Guided Bin Packing

Diffusion models enable the integration of custom cost functions during sampling. A straightforward mechanism for imposing hard constraints is *inpainting*, where placements subject to fixed preferences are overwritten after each denoising step. This operation can be expressed as

$$\mathbf{x}_{t-1} = (\mathbf{1} - \mathbf{M}) \odot \tilde{\mathbf{x}}_{t-1} + \mathbf{M} \odot \mathbf{y}_0, \quad (3)$$

where $\mathbf{M}$ is a binary mask indicating cuboids with hard inpainting constraints, preserving their values during diffusion.

Algorithm 1 PROJECTION OPERATOR (P_C)

Require: $\mathbf{C} \in \mathbb{R}^{L \times 3}$ (centroids), $\mathbf{D} \in \mathbb{R}^{L \times 3}$ (dimensions)
Ensure: $\hat{\mathbf{C}} \in \mathbb{R}^{L \times 3}$ (adjusted centroids, no overlaps)

- 1: Initialize $\hat{\mathbf{C}} \leftarrow \mathbf{0}$
- 2: $\mathcal{B} \leftarrow \emptyset$ ▷ Placed cuboids indices
- 3: $\mathcal{U} \leftarrow \{1, \dots, L\}$ ▷ Unplaced indices
- 4: **Place the closest cuboid to the origin at the origin:**
- 5: $i^* \leftarrow \arg \min_{i \in \mathcal{U}} \|\mathbf{C}_i\|_2$
- 6: $\hat{\mathbf{C}}_{i^*} \leftarrow (0, 0, 0)$
- 7: $\mathcal{B} \leftarrow \mathcal{B} \cup \{i^*\}$, $\mathcal{U} \leftarrow \mathcal{U} \setminus \{i^*\}$
- 8: **Place other cuboids:**
- 9: **while** $\mathcal{U} \neq \emptyset$ **do**
- 10: $\mathcal{Q} \leftarrow$ candidate points generated from $\mathcal{B}$
- 11: $\mathcal{F} \leftarrow \emptyset$
- 12: **for** $(j, \mathbf{q}) \in \mathcal{U} \times \mathcal{Q}$ **do**
- 13: $\mathbf{p} \leftarrow \text{ProjectCuboid}(j, \mathbf{q})$
- 14: **if** $\text{CollisionFree}(\mathbf{p})$ **then**
- 15: $\mathcal{F} \leftarrow \mathcal{F} \cup \{(j, \mathbf{p}, \|\mathbf{p} - \mathbf{C}_j\|_2)\}$
- 16: **end if**
- 17: **end for**
- 18: **if** $\mathcal{F}$ is empty **then**
- 19: **break** ▷ no feasible projections
- 20: **else**
- 21: $(j^*, \mathbf{p}^*) \leftarrow \arg \min_{(j, \mathbf{p}, d) \in \mathcal{F}} d$
- 22: $\hat{\mathbf{C}}_{j^*} \leftarrow \mathbf{p}^*$ ▷ Assign $\mathbf{p}^*$ to cuboid j^*
- 23: $\mathcal{B} \leftarrow \mathcal{B} \cup \{j^*\}$, $\mathcal{U} \leftarrow \mathcal{U} \setminus \{j^*\}$
- 24: **end if**
- 25: **end while**
- 26: **return** $\hat{\mathbf{C}}$

$\tilde{\mathbf{x}}_{t-1}$ is the unconstrained diffusion output, and $\mathbf{y}_0$ provides the reference values for enforcing these constraints.

Another popular approach is to guide the sampling towards optimizing a specific cost function. The transition probability from x_t to x_{t-1} , conditioned on time t and cost C , given as,

$$\log p(x_{t-1} | x_t, t, C) = \log \mathcal{N}(z; \mu_t + \Sigma_t g, \Sigma_t), \quad (4)$$

where the guidance term $g = \nabla_{x_{t-1}} \log p(C | x_{t-1})|_{x_{t-1}=\mu_t}$ is the gradient of the log-likelihood of the cost C with respect to x_{t-1} , evaluated at $x_{t-1} = \mu_t$.

Assuming the cost functions $c_i \in \mathcal{C}$ are locally differentiable, the guidance term g becomes:

$$g = - \sum_i \lambda_i \nabla_{x_{t-1}} c_i(x_{t-1})|_{x_{t-1}=\mu_t}, \quad (5)$$

where λ_i are scalar weights selected via grid-search for each cost. Our method only requires locally differentiable costs, making it robust to approximate formulations as similar local gradients yield comparable behavior. Sensitivity to guidance weights is mitigated through GPU-parallelized diffusion sampling with grid search, where $\lambda_i \in [1, 25]$.

D. Test-time Preference Integration Using LLMs

We allow users to specify preferences in natural language, which are categorized as hard or soft. Hard preferences

Algorithm 2 DiffusionPack

Require: Number of Samples B , Dimensions c , Denoising Model ε_θ , Total steps N , Guidance costs G , Guidance weights λ , Projection operator $\mathcal{P}_C$, Hard preferences H

Ensure: Final feasible cuboid configuration x_0

```
1: Candidates  $\leftarrow []$ 
2: for  $i = 1$  to  $B$  do
3:   Sample noise  $x_T \sim \mathcal{N}(0, I)$ 
4:   for  $t = N$  to 1 do
5:      $\mu_t \leftarrow \frac{1}{\sqrt{\alpha_t}} \left( x_t - \frac{1-\alpha_t}{\sqrt{1-\alpha_t}} \varepsilon_\theta(x_t, c, t) \right)$ 
6:      $g \leftarrow - \sum_k \lambda_k \nabla_{x_{t-1}} G_k(x_{t-1}) \Big|_{x_{t-1}=\mu_t}$ 
7:      $\mu_t \leftarrow \mu_t + \Sigma_t g$ 
8:      $x_{t-1} \sim \mu_t + \sqrt{\tilde{\beta}_t} \cdot \mathcal{N}(0, I)$ 
9:      $x_{t-1} \leftarrow \text{Inpaint}(x_{t-1}, H)$ 
10:    if  $t < T_{\text{proj}}$  then
11:       $x_{t-1} \leftarrow \mathcal{P}_C(x_{t-1})$  (Algorithm 1)
12:    end if
13:  end for
14:  Candidates[ $i$ ]  $\leftarrow x_0^{(i)}$ 
15: end for
16: Candidates  $\leftarrow \text{STABILIZE}(\text{Candidates})$ 
17: Evaluate: For each candidate  $x_0^{(i)}$ , compute Utility Rate(UR), Packing Density(PD), and Item Count.
18: Select Objective  $\in \{\text{UR}, \text{PD}, \text{ItemCount}\}$ 
19: return  $x_0^* \leftarrow \arg \max_{x_0^{(i)} \in \text{Candidates}} \text{Objective}(x_0^{(i)})$ 
```

are enforced via inpainting after each denoising step using predefined rules (Eq. 3), while soft preferences are formulated as differentiable cost functions for gradient-based guidance (Eq. 4, 5). A pretrained LLM translates these natural language inputs (e.g., “Cuboid 1 should be on top of Cuboid 4” or “heavier cuboids should be at the bottom”) into executable Python code implementing the corresponding cost functions and/or inpainting rules (Fig. 2(3)). Users can inspect and iteratively refine the generated code, ensuring interpretable and controllable preference integration. The LLM serves purely as a convenience layer that lowers the barrier to specifying constraints; it is not a core algorithmic dependency. Without it, users would need to manually implement equivalent Python functions. We use a pretrained GPT-4o model (temperature = 0.2), without fine-tuning or in-context learning. Prompts are designed to (1) enforce a machine-parsable output format and (2) encode scene-specific conventions (e.g., coordinate systems and library usage). All prompts are included in the released code.

IV. EXPERIMENTS AND RESULTS

This section evaluates **DiffusionPack** in synthetic and real settings, assessing its effectiveness as a packing planner on both in-distribution and out-of-distribution datasets under unconstrained and constrained conditions. Our experiments are designed to answer three main questions:

- **Q1.** How effectively does the method generate valid,

stable, and space-efficient 3D packing configurations?

- **Q2.** How robustly can the method incorporate custom constraints introduced at test time?
- **Q3.** What is the individual contribution of the diffusion module and the projection operator?

A. Dataset

We evaluate our approach on two widely recognized datasets: **Cutting Stock Dataset (CSD)**. Following [6], we generate compact packing layouts by recursively partitioning the container into subregions with a bias toward longer dimensions. This dataset enables training on high-density configurations and supports an upper bound of 100% occupancy, which provides a clear benchmark for packing efficiency.

Randomly Sampled Dataset (RSD). To assess generalization, we evaluate on an unseen dataset of randomly sampled cuboids. Each cuboid dimension is drawn uniformly within a $L \times W \times H$ container, and is further constrained to $\frac{\min(L, W, H)}{10} \leq l_t, w_t, h_t \leq \frac{\min(L, W, H)}{2}$ ensuring that the dataset excludes degenerate tiny items as well as oversized slabs that trivialize the packing task. Dimensions are normalized to a unit container for scale invariance across datasets. Note that RSD does not provide an upper bound, and a complete packing of all items may not always be feasible.

B. Model Details and Training

The denoiser predicts Gaussian noise using a mean squared error objective, following DDPM [36]. We employ a DiT-style [37] transformer adapted for 3D cuboid embeddings. The diffusion model takes noisy cuboid centroids in $\mathbb{R}^3$, diffusion timestep, and cuboid dimensions in $\mathbb{R}^3$ (via cross-attention) as input, and outputs estimated noise in $\mathbb{R}^3$. All inputs are projected via an MLP. The timestep embeddings are incorporated using FiLM-style modulation [38]. No positional encoding is applied, ensuring permutation invariance across unordered cuboid sets. The architecture (Fig. 2) comprises six DiT layers with a hidden size of 256 and 8 attention heads, maintaining global context while preserving set exchangeability. We use one million samples from the Cutting Stock Dataset, drawn with replacement over five million steps, with a 70:20:10 train-validation-test split. We optimize with AdamW (3e-5 learning rate, batch size 2048). Inference generates 512 candidate configurations using 250-step DDPM sampling, requiring up to only 1 GB of VRAM. All experiments are conducted on an Intel i9 CPU and an NVIDIA RTX 6000 Ada GPU (48 GB VRAM).

C. Evaluation

We evaluate *DiffusionPack* in two complementary settings: (i) unconstrained bin packing measuring geometric efficiency, and (ii) constrained bin packing requiring integration of human-specified preferences. Performance is assessed using Packing Density (PD), Utilization Rate (UR), and the number of placed cuboids. Packing Density is defined as $\text{PD} = \frac{\sum_{i=1}^N V_i}{V_{\text{container}}}$, while Utilization Rate captures both volume usage and stacking height, $\text{UR} = \frac{\sum_{i=1}^N V_i}{A_{\text{base}} \cdot H_{\text{max}}}$, where A_{base} is the container’s base area and H_{max} is the maximum packing

TABLE I: Performance comparison across different methods and settings. We report the Utility Rate (UR), Packing Density (PD), Average number of cuboids packed (ACP), and inference Time (s). Results are reported as mean $\pm$ standard deviation over 5 seeds. Average running time is reported for each baseline on 32-object instances for runtime comparison.

Method	Time (s)	CSD-16			CSD-32			RSD-16			RSD-32		
		UR	PD	ACP	UR	PD	ACP	UR	PD	ACP	UR	PD	ACP
First Fit (EP) [11]	0.01	56.7 $\pm$ 0.3	44.8 $\pm$ 0.3	13.2	53.2 $\pm$ 0.4	43.7 $\pm$ 0.7	22.4	57.4 $\pm$ 0.3	54.4 $\pm$ 0.2	12.8	67.5 $\pm$ 0.2	67.1 $\pm$ 0.2	14.3
Best Fit (EP) [11]	0.01	78.5 $\pm$ 0.4	54.2 $\pm$ 1.0	13.6	72.4 $\pm$ 0.2	54.6 $\pm$ 0.9	23.1	69.0 $\pm$ 0.2	65.6 $\pm$ 0.2	12.1	82.2 $\pm$ 0.2	82.7 $\pm$ 0.2	14.5
Key Point [34]	0.04	61.3 $\pm$ 1.0	53.5 $\pm$ 1.5	13.7	60.1 $\pm$ 1.2	53.5 $\pm$ 1.5	22.8	63.0 $\pm$ 1.1	66.4 $\pm$ 0.8	12.4	69.7 $\pm$ 0.3	69.9 $\pm$ 0.4	17.9
BRKGA [14]	1.20	58.9 $\pm$ 0.9	45.4 $\pm$ 0.4	13.2	58.1 $\pm$ 0.9	45.3 $\pm$ 1.0	22.0	67.4 $\pm$ 0.2	57.6 $\pm$ 0.9	12.6	71.1 $\pm$ 0.2	68.3 $\pm$ 0.5	17.1
DBLF + GA [35]	1.44	94.5 $\pm$ 0.4	82.2 $\pm$ 0.4	15.9	70.8 $\pm$ 0.3	60.3 $\pm$ 0.4	26.8	71.4 $\pm$ 0.5	58.1 $\pm$ 0.4	12.1	67.5 $\pm$ 0.3	65.3 $\pm$ 0.2	17.5
RCQL [19]	1.25	87.1 $\pm$ 0.1	71.8 $\pm$ 0.1	13.6	62.6 $\pm$ 0.1	59.3 $\pm$ 0.0	23.5	69.1 $\pm$ 0.1	57.8 $\pm$ 0.1	13.1	69.6 $\pm$ 0.0	61.3 $\pm$ 0.0	18.2
Ranked-Reward [6]	–	49.3 $\pm$ 5.0	53.8 $\pm$ 5.1	13.5	50.9 $\pm$ 2.3	57.6 $\pm$ 0.3	22.8	61.9 $\pm$ 0.1	64.3 $\pm$ 0.3	15.2	71.8 $\pm$ 0.1	72.0 $\pm$ 0.0	19.2
GUROBI	63.22	97.1 $\pm$ 0.0	81.8 $\pm$ 0.1	15.6	77.6 $\pm$ 0.1	69.3 $\pm$ 0.0	26.4	74.8 $\pm$ 0.1	67.9$\pm$0.0	13.7	73.6 $\pm$ 0.0	68.3 $\pm$ 0.0	18.9
Diffusion-Only	8.05	48.9 $\pm$ 0.2	33.1 $\pm$ 0.4	10.1	43.4 $\pm$ 1.6	34.7 $\pm$ 0.6	12.4	47.2 $\pm$ 0.4	34.2 $\pm$ 0.2	10.5	49.5 $\pm$ 1.5	35.1 $\pm$ 0.2	12.1
Diffusion-CP	9.95	99.2 $\pm$ 0.0	90.1 $\pm$ 0.2	15.9	93.5 $\pm$ 0.2	89.1 $\pm$ 0.4	28.7	68.3 $\pm$ 0.2	64.0 $\pm$ 0.5	12.9	79.6 $\pm$ 0.2	70.7 $\pm$ 0.1	18.8
Diffusion-EP	8.52	99.1 $\pm$ 0.0	90.1 $\pm$ 0.2	15.9	93.5 $\pm$ 0.2	89.5 $\pm$ 0.4	29.1	68.1 $\pm$ 0.2	65.9 $\pm$ 0.6	13.5	80.8 $\pm$ 0.2	71.8 $\pm$ 0.2	19.2
Diffusion-EMS	6.22	99.8$\pm$0.1	90.1 $\pm$ 0.2	16.0	99.4$\pm$0.2	90.1$\pm$0.1	31.8	75.8$\pm$0.1	67.3 $\pm$ 0.3	13.6	85.7$\pm$0.2	76.3$\pm$0.2	19.8

height. These metrics together reflect spatial efficiency and physical feasibility, which are essential in robotic palletizing and warehouse automation. Results are reported over five seeds for both 16 and 32 cuboids (CSD-16/32, RSD-16/32).

1) *3D Bin Packing Without Preferences*: We first evaluate **DiffusionPack** in the unconstrained setting. Table I compares our method against heuristic, genetic, reinforcement learning, and mixed-integer programming baselines, including RCQL, the strongest publicly available RL method for offline 3D bin packing. All baselines are adapted to produce stable configurations. For the commercial solver GUROBI, stability is hard to formulate through linear programs, so we apply a similar post-stabilization step as DiffusionPack for fair comparison. As illustrated in Fig. 4 (Panels 1–5), the model captures global spatial structure and produces compact arrangements for both full and partial packing sequences. DiffusionPack (EMS) produces a complete solution in roughly 6 s. Since mixed-integer solvers like Gurobi are exact and can run for infinite time, we evaluate GUROBI with 10 $\times$ more time to assess whether increased compute reduces the performance gap. GUROBI ran on 32 CPU cores with built-in Gurobi CPU parallelization, while DiffusionPack used a single CPU core and one GPU. Heuristic methods remain faster but consistently yield lower PD and UR. Other RL approaches [5], [6], [16], [17] are excluded due to the absence of verified, publicly available implementations. As shown in Table I, DiffusionPack achieves significantly higher utilization and packing density, including strong generalization on RSD datasets.

TABLE II: Performance on custom human preferences

Object Count	DiffusionCCSP		DiffusionPack		DiffusionPack-LLM	
	SR	UR	SR	UR	SR	UR
5 Objects	52.0	48.4	71.0	68.2	66.0	62.1
16 Objects	5.0	45.6	66.0	72.4	59.0	79.0
32 Objects	0.0	–	62.0	78.7	53.0	77.7

2) *3D Bin Packing With Custom Human Preferences*: We next evaluate the ability of **DiffusionPack** to incorporate user-defined constraints expressed at test time. We define

Success Rate (SR) as the percentage of packings that satisfy all specified constraints. We evaluate zero-shot performance on 100 unseen instances across diverse scenarios, including preferred placement, object grouping, and relative spatial constraints (ordering, proximity, separation). Fig. 4 illustrates representative results: Panels A and B show strict enforcement of hard constraints via inpainting, and Panels C and D demonstrate the effect of soft constraints implemented using gradient-based guidance. Table II compares DiffusionPack with Diffusion-CCSP [31]. DiffusionPack uses manually defined cost functions and inpainting rules, while DiffusionPack-LLM employs LLM-generated ones. DiffusionPack attains higher success rates, scales to 32 objects (vs. 5-6 for CCSP), and supports multiple constraints without retraining. The LLM-based variant achieves comparable performance despite using automatically generated code.

D. Ablation Studies

TABLE III: Ablation on applying projection after diffusion steps. The reported metric is Utility Rate over 5 seeds.

Method	CSD-16 (%)	CSD-32 (%)	RSD-16 (%)	RSD-32 (%)
No projections	48.9 $\pm$ 0.2	43.4 $\pm$ 1.6	47.2 $\pm$ 0.4	49.5 $\pm$ 1.5
Each step	63.3 $\pm$ 0.1	58.8 $\pm$ 0.1	52.4 $\pm$ 0.2	55.5 $\pm$ 0.2
Last-10 steps	93.6 $\pm$ 0.0	91.1 $\pm$ 0.1	58.2 $\pm$ 0.0	63.2 $\pm$ 0.0
Last-5 steps	98.4 $\pm$ 0.1	97.7 $\pm$ 0.0	62.3 $\pm$ 0.1	69.3 $\pm$ 0.1
Last-2 steps	99.3 $\pm$ 0.0	98.2 $\pm$ 0.1	64.3 $\pm$ 0.1	74.2 $\pm$ 0.1
Final step	99.8 $\pm$ 0.1	99.4 $\pm$ 0.2	75.8 $\pm$ 0.1	85.7 $\pm$ 0.2

We perform ablation studies to quantify the contribution of each component in **DiffusionPack**, focusing on (i) the effect of projection timing on feasibility and packing efficiency, and (ii) the impact of projection-point generation strategy on final performance.

Ablations on Projection Timing: We evaluate four strategies for applying the projection operator during sampling:

- 1) **No projection (baseline)**: Diffusion sampler without geometric correction.
- 2) **Every step**: Projection applied at all t .
- 3) **Last (k) steps**: Projection applied only during the final (k) denoising iterations.

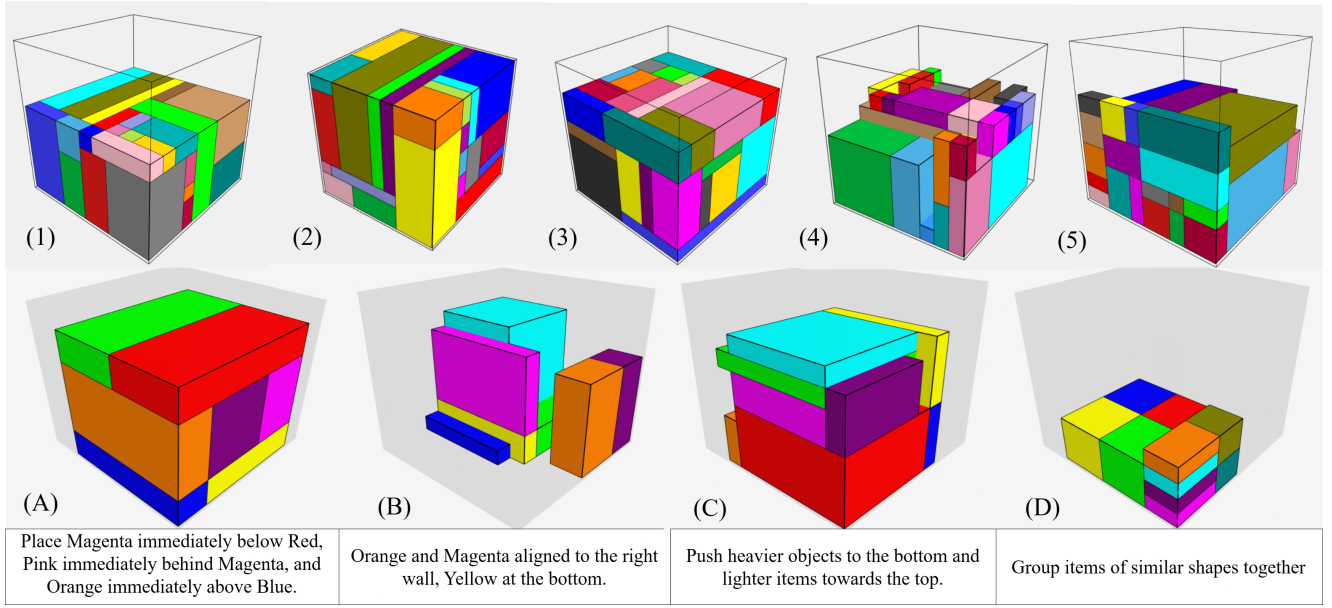


Fig. 4: **Packing results for up to 32 cuboids** Our method aims to produce compact arrangements, even when the total cuboid volume does not fully span the container, yielding dense configurations. **Packing results with human-defined preferences** Panels A and B apply hard preferences via inpainting, while Panels C and D use guidance functions to steer the packing process iteratively. We present a limited set of cuboids here to keep the illustration clear and concise.

4) **Final step only:** Projection applied once at $t = 0$.

Table III summarizes the results. Diffusion-only samples are often infeasible, so for fair comparison, we remove highly overlapping cuboids and project the remaining items downward to obtain a valid configuration. Applying projection at every step overconstrains sampling and reduces diversity, while early projection is ineffective due to high positional variance at large t . Final-step projection performs best, enabling global structure formation with minimal feasibility correction.

Ablations on Projection-Point Selection: We compare three point-based heuristics, repurposed as projection points in our projection operator. Refer to Table I.

- 1) **Corner Points (CP):** Axis-aligned corner candidates derived from placed cuboids [33].
- 2) **Extreme Points (EP):** Contact-based points formed by intersections of placed cuboids and container [11].
- 3) **Empty Maximal Spaces (EMS):** Axis-aligned maximal free volumes representing remaining unoccupied regions [39].

EMS yields the strongest performance because it directs projections into large unoccupied spaces with high structural freedom. EP explores fewer candidate placements, and CP behaves greedily by overfitting to corner-aligned placements. These trends are reflected in Table I, where EMS consistently produces the most space-efficient arrangements.

E. Real-World Physical Evaluation

We validate **DiffusionPack** on a real robotic platform to assess physical feasibility and execution robustness. Experiments are performed on a Franka FR3 with a suction gripper, packing object sets from six standardized sizes. The robot executes placements predicted by our pipeline sequentially



Fig. 5: Final pallet configurations from real-world experiments, showing compact and stable placements.

in depth order to avoid collisions. At the start of each trial, objects are localized using AprilTags. To account for perception and actuation uncertainties, all cuboid dimensions are inflated by 1 cm during planning. Each cuboid is picked from its estimated pose, positioned above the target pose, and lowered with a guarded motion. A trial is deemed successful if no collision occurs and the final positional error is below 2 cm. Across ten trials, the system achieved an 80% success rate. The resulting pallet configurations (Fig. 5) demonstrate compact, stable arrangements.

V. CONCLUSION

We introduced **DiffusionPack**, a diffusion-guided framework for offline 3D bin packing that jointly reasons over all item placements in continuous space and integrates user-defined constraints at test time. The method combines global layout prediction via diffusion with a geometric projection operator that enforces feasibility, yielding compact and stable packings across synthetic, out-of-distribution, and real-world settings. Experiments show that DiffusionPack consistently outperforms heuristic, learning, and optimization-based base-

lines while aligning with human preferences.

Limitations and Future Works: Despite its effectiveness, **DiffusionPack** has several limitations. The current formulation assumes fixed axis-aligned orientations. LLM-based preference translation can yield inaccurate or non-differentiable costs, reducing reliability. Unclear or conflicting user constraints may lead to unsatisfiable preferences. Training becomes computationally intensive, requiring a multi-GPU setup for more than 32 items. Future work includes extending to orientation-aware packing and adopting scalable attention mechanisms for larger item sets. Automated preference validation and conflict resolution for LLM-generated code, along with learned projection operators to reduce reliance on heuristics, are promising directions. Additionally, accelerating inference through improved diffusion samplers remains a key next step.

REFERENCES

- [1] X. Schepler, A. Rossi, E. Gurevsky, and A. Dolgui, "Solving robust bin-packing problems with a branch-and-bound approach," *European Journal of Operational Research*, vol. 297, no. 3, pp. 831–843, 2022.
- [2] G. Dósa, "The tight bound of first fit decreasing bin-packing algorithm is $\text{ff}(i) \leq 11/9 \text{opt}(i) + 6/9$," in *Proceedings of the First International Conference on Combinatorics, Algorithms, Probabilistic and Experimental Methodologies*, ser. ESCAPE'07. Berlin, Heidelberg: Springer-Verlag, 2007, p. 1–11.
- [3] D. S. Johnson, "Fast algorithms for bin packing," *Journal of Computer and System Sciences*, vol. 8, no. 3, pp. 272–314, 1974.
- [4] H. Zhao, Q. She, C. Zhu, Y. Yang, and K. Xu, "Online 3d bin packing with constrained deep reinforcement learning," 2022. [Online]. Available: <https://arxiv.org/abs/2006.14978>
- [5] J. Zhang, B. Zi, and X. Ge, "Attend2pack: Bin packing through deep reinforcement learning with attention," *arXiv preprint arXiv:2107.04333*, 2021.
- [6] A. Laterre, Y. Fu, M. K. Jabri, A.-S. Cohen, D. Kas, K. Hajjar, T. S. Dahl, A. Kerkeni, and K. Beguir, "Ranked reward: Enabling self-play reinforcement learning for combinatorial optimization," *arXiv preprint arXiv:1807.01672*, 2018.
- [7] S. Martello and P. Toth, "Generalized assignment problems," in *Algorithms and Computation*, T. Ibaraki, Y. Inagaki, K. Iwama, T. Nishizeki, and M. Yamashita, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 1992, pp. 351–369.
- [8] H. Dyckhoff, "A new linear programming approach to the cutting stock problem," *Operations Research*, vol. 29, pp. 1092–1104, 12 1981.
- [9] H. Cambazard and B. O'Sullivan, "Propagating the bin packing constraint using linear programming," vol. 6308, 2010, pp. 129–136.
- [10] K. Salem and Y. Kieffer, *New Symmetry-less ILP Formulation for the Classical One Dimensional Bin-Packing Problem*, 2020, pp. 423–434.
- [11] T. G. Crainic, G. Perboli, and R. Tadei, "Extreme point-based heuristics for three-dimensional bin packing," *INFORMS J. on Computing*, vol. 20, no. 3, p. 368–384, July 2008.
- [12] T. Fanslau and A. Bortfeldt, "A tree search algorithm for solving the container loading problem," *INFORMS J. on Computing*, vol. 22, no. 2, p. 222–235, Apr. 2010.
- [13] J. Kang, I. Moon, and H. Wang, "A hybrid genetic algorithm with a new packing strategy for the three-dimensional bin packing problem," *Applied Mathematics and Computation*, vol. 219, p. 1287–1299, 2012.
- [14] H. Prasetyo, A. Putri, and G. Fauza, "Biased random key genetic algorithm design with multiple populations to solve capacitated vehicle routing problem with time windows," vol. 1977, 06 2018, p. 020052.
- [15] T. G. Crainic, G. Perboli, and R. Tadei, "Ts2pack: A two-level tabu search for the three-dimensional bin packing problem," *European Journal of Operational Research*, vol. 195, pp. 744–760, 06 2009.
- [16] H. Hu, X. Zhang, X. Yan, L. Wang, and Y. Xu, "Solving a new 3d bin packing problem with deep reinforcement learning method," *arXiv preprint arXiv:1708.05930*, 2017.
- [17] L. Duan, H. Hu, Y. Qian, Y. Gong, X. Zhang, J. Wei, and Y. Xu, "A multi-task selected learning approach for solving 3d flexible bin packing problem," in *Proceedings of the 18th International Conference on Autonomous Agents and MultiAgent Systems*, 2019, pp. 1386–1394.
- [18] Y. Jiang, Z. Cao, and J. Zhang, "Solving 3d bin packing problem via multimodal deep reinforcement learning," in *Proceedings of the 20th International Conference on Autonomous Agents and MultiAgent Systems*, ser. AAMAS '21. Richland, SC: International Foundation for Autonomous Agents and Multiagent Systems, 2021, p. 1548–1550.
- [19] D. Li, Z. Gu, Y. Wang, C. Ren, and F. C. Lau, "One model packs thousands of items with recurrent conditional query learning," *Knowledge-Based Systems*, vol. 235, p. 107683, 2022.
- [20] Z. Boliang, Y. Yao, H. Kan, and W. Luo, "A gan-based genetic algorithm for solving the 3d bin packing problem," *Scientific Reports*, vol. 14, 04 2024.
- [21] Z. Sun and Y. Yang, "Difusco: graph-based diffusion solvers for combinatorial optimization," in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, ser. NIPS '23. Red Hook, NY, USA: Curran Associates Inc., 2023.
- [22] S. Sanokowski, S. Hochreiter, and S. Lehner, "A diffusion model framework for unsupervised neural combinatorial optimization," in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML'24. JMLR.org, 2024.
- [23] W. Liu, Y. Du, T. Hermans, S. Chernova, and C. Paxton, "Strucdiffusion: Language-guided creation of physically-valid structures using unseen objects," in *Robotics: Science and Systems*, 2023.
- [24] Q. A. Wei, S. Ding, J. J. Park, R. Sajjani, A. Poulenard, S. Sridhar, and L. Guibas, "Lego-net: Learning regular rearrangements of objects in rooms," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 19037–19047.
- [25] M. Wu, F. Zhong, Y. Xia, and H. Dong, "Targf: learning target gradient field to rearrange objects without explicit goal specification," in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, ser. NIPS '22, 2022.
- [26] T. Xue, M. Wu, L. Lu, H. Wang, H. Dong, and B. Chen, "Learning gradient fields for scalable and generalizable irregular packing," in *SIGGRAPH Asia 2023 Conference Papers*, ser. SA '23. New York, NY, USA: Association for Computing Machinery, 2023.
- [27] P. Wojciechowski, K. Subramani, A. Velasquez, and B. Caskurlu, "Priority-based bin packing with subset constraints," *Discrete Applied Mathematics*, vol. 342, pp. 64–75, 2024.
- [28] S. Erbayrak, V. Özkır, and U. Mahir Yıldırım, "Multi-objective 3d bin packing problem with load balance and product family concerns," *Computers & Industrial Engineering*, vol. 159, p. 107518, 2021.
- [29] W. Zhu, S. Chen, M. Dai, and J. Tao, "Solving a 3d bin packing problem with stacking constraints," *Computers & Industrial Engineering*, vol. 188, p. 109814, 2024.
- [30] H. Zhao, Y. Yu, and K. Xu, "Learning efficient online 3d bin packing on packing configuration trees," in *International Conference on Learning Representations*, 2022.
- [31] Z. Yang, J. Mao, Y. Du, J. Wu, J. B. Tenenbaum, T. Lozano-Pérez, and L. P. Kaelbling, "Compositional diffusion-based continuous constraint solvers," in *Conference on Robot Learning*, 2023, pp. 3242–3265.
- [32] J. K. Christopher, S. Baek, and N. Fioretto, "Constrained synthesis with projected diffusion models," *Advances in Neural Information Processing Systems*, vol. 37, pp. 89 307–89 333, 2024.
- [33] S. Martello, D. Pisinger, and D. Vigo, "The three-dimensional bin packing problem," *Oper. Res.*, vol. 48, no. 2, p. 256–267, Mar. 2000.
- [34] X. Tian, pengzhan qu, and H. Qi, "Key point is key in resolving the offline three-dimensional bin packing problem," 2024. [Online]. Available: <https://openreview.net/forum?id=4Y4hPWUppN>
- [35] K. Karabulut and M. M. undefinednceoğlu, "A hybrid genetic algorithm for packing in 3d with deepest bottom left with fill method," in *Proceedings of the Third International Conference on Advances in Information Systems*, 2004, p. 441–450.
- [36] J. Ho, A. Jain, and P. Abbeel, "Denosing diffusion probabilistic models," *Advances in neural information processing systems*, vol. 33, pp. 6840–6851, 2020.
- [37] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2023, pp. 4195–4205.
- [38] E. Perez, F. Strub, H. De Vries, V. Dumoulin, and A. Courville, "Film: Visual reasoning with a general conditioning layer," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 32, no. 1, 2018.
- [39] C. T. Ha, T. T. Nguyen, L. T. Bui, and R. Wang, "An online packing heuristic for the three-dimensional container loading problem in dynamic environments and the physical internet," in *20th European Conference, EvoApplications*, vol. 10200, 2017, pp. 140–155.